

Q3 - Garantia de regra de negócio (TRIGGER com AUDITORIA)

Banco: sakila

REGRA DE NEGÓCIO:

Um exemplar (inventory_id) não pode ser locado novamente, enquanto a locação anterior desse mesmo exemplar ainda estiverem.

IMPORTÂNCIA

O schema padrão do Sakila não possui nenhuma constraint que impeça a "dupla locação" de um exemplar físico. Sem essa regra, seria possível registrar a locação de uma mídia que já está com outro cliente, gerando inconsistência de estoque e conflito de atendimento. Essa validação garante integridade operacional do processo de locação.

1) TABELA DE LOG / AUDITORIA

```
USE sakila;

DROP TABLE IF EXISTS log_auditoria_locacao;

CREATE TABLE log_auditoria_locacao (
    id_log            INT AUTO_INCREMENT PRIMARY KEY,
    data_hora         DATETIME DEFAULT CURRENT_TIMESTAMP,
    usuario           VARCHAR(100),
    operacao          VARCHAR(50),
    tabela_afetada    VARCHAR(50),
    id_inventory       INT,
    id_customer        INT,
    descricao_problema VARCHAR(255)
) ENGINE=MyISAM;
```

2) TRIGGER: bloqueia locação de exemplar já em aberto

```
DROP TRIGGER IF EXISTS trg_bloqueia_locacao_duplicada$$

CREATE TRIGGER trg_bloqueia_locacao_duplicada
BEFORE INSERT ON rental
FOR EACH ROW
BEGIN
    DECLARE v_locacoes_abertas INT;

    SELECT COUNT(*) INTO v_locacoes_abertas
    FROM rental
    WHERE inventory_id = NEW.inventory_id
    AND return_date IS NULL;

    IF v_locacoes_abertas > 0 THEN

        INSERT INTO log_auditoria_locacao
        (usuario, operacao, tabela_afetada, id_inventory, id_customer, descricao_problema)
        VALUES
        (CURRENT_USER(), 'INSERT BLOQUEADO', 'rental', NEW.inventory_id, NEW.customer_id,
        CONCAT('Tentativa de locar o exemplar ', NEW.inventory_id,
        ' que já possui locação em aberto (sem devolução registrada)'));

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Operação bloqueada: este exemplar já está locado e ainda não foi devolvido.';

    END IF;
END$$
```

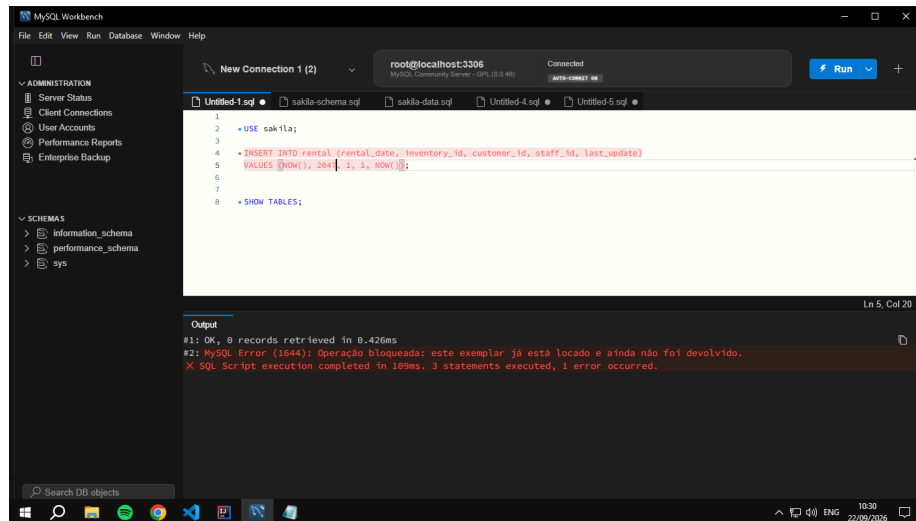
3) TESTES / DEMONSTRAÇÃO

3.1) Localizar um exemplar que está atualmente em aberto (sem devolução)

```
SELECT rental_id, inventory_id, customer_id, rental_date
FROM rental
WHERE return_date IS NULL
LIMIT 1;
```

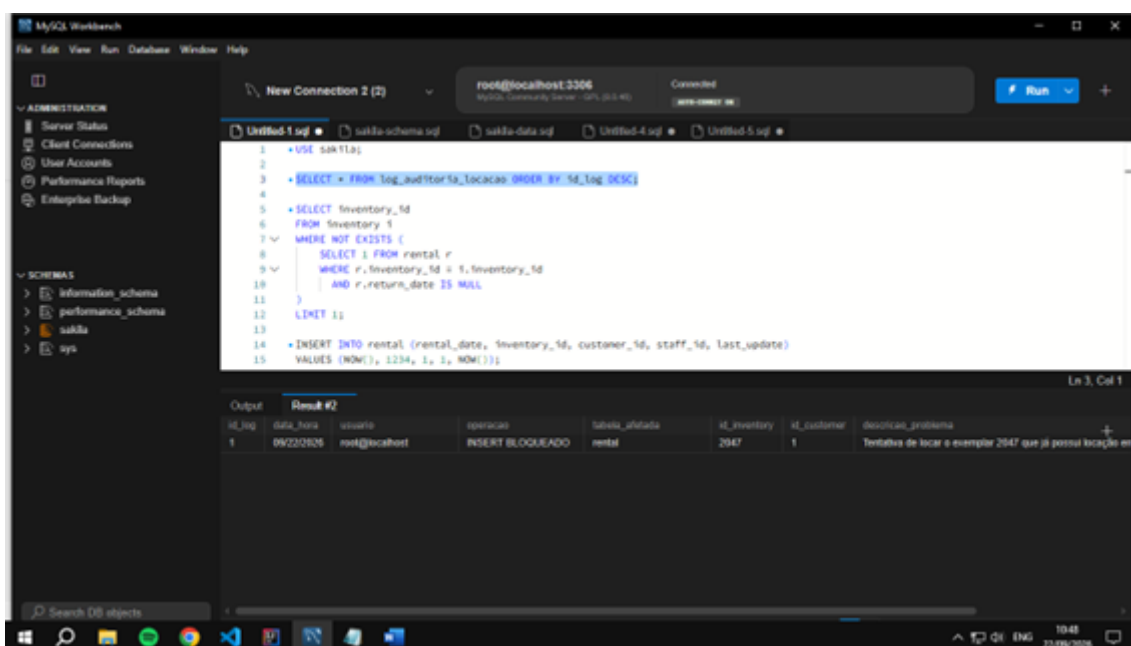
3.2) TESTE 1 - Tentativa inválida (deve ser BLOQUEADA e registrada no log)

```
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id, last_update)
VALUES (NOW(), inventory_id, 1, 1, NOW());
```



3.3) Conferir que a tentativa foi registrada na auditoria

```
SELECT * FROM log_auditoria_locacao ORDER BY id_log DESC;
```



3.4) TESTE 2 - Locação válida (deve funcionar normalmente)

```
SELECT inventory_id
FROM inventory i
WHERE NOT EXISTS (
    SELECT 1 FROM rental r
    WHERE r.inventory_id = i.inventory_id
        AND r.return_date IS NULL
)
LIMIT 1;
```

```
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id, last_update)
VALUES (NOW(), <inventory_id_disponivel>, 1, 1, NOW());
```

